



**Politechnika
Śląska**

Dokumentacja projektowa

Zarządzanie Systemami Informatycznymi

*Symulacja zarządzania projektem wytwarzania oprogramowania
w systemie Jira*

Kierunek: Informatyka

Członkowie zespołu:

Tomasz Solarz

Andrzej Machnik

Wiktor Jarosz

Gliwice, 2025/2026

Spis treści

1	Wprowadzenie	2
1.1	Role w projekcie	2
1.2	Cel projektu	2
2	Założenia projektowe	3
2.1	Założenia techniczne i nietechniczne	3
2.2	Stos technologiczny	3
2.3	Oczekiwane rezultaty projektu	3
3	Realizacja projektu	4
3.1	Konfiguracja początkowa i zespół	4
3.2	Struktura zadań i planowanie wydań	5
3.3	Backlog i planowanie Sprintów	7
3.4	Automatyzacje	8
3.5	Przebieg Sprintów i spotkania codzienne (Stand-up)	10
3.6	Raportowanie i analiza efektywności	12
4	Wnioski	16
5	Bibliografia	17

1 Wprowadzenie

1.1 Role w projekcie

W ramach projektu przyjęto następujący podział ról w zespole projektowym (Scrum Team):

- **Tomasz Solarz** – Scrum Master / Developer
- **Andrzej Machnik** – Developer
- **Wiktor Jarosz** – Product Owner / Developer

Ze względu na niewielki, trzyosobowy zespół, każdy z członków aktywnie uczestniczył w wytwarzaniu oprogramowania na stanowisku Developera, jednocześnie pełniąc kluczowe role wspierające proces zarządzania (Scrum Master, Product Owner).

1.2 Cel projektu

Celem projektu była symulacja małego projektu z wytwarzania oprogramowania przy użyciu platformy Jira w ramach zwinnej metodyki Scrum. Naszym zadaniem było przeprowadzenie procesu twórczego na przestrzeni kilku sprintów oraz praktyczne wykorzystanie i zaprezentowanie różnorodnych funkcjonalności oferowanych przez system Jira. Skupiliśmy się na takich elementach jak planowanie sprintów, zarządzanie backlogiem, automatyzacje przepływu pracy, zarządzanie wydaniem oraz generowanie raportów postępów pracy zespołu.

2 Założenia projektowe

2.1 Założenia techniczne i nietechniczne

- Organizacja pracy w metodyce zwinnej (Agile/Scrum).
- Zdefiniowanie przejrzystego przepływu pracy opartego o kolumny: To Do, In Progress, In Review oraz Done.
- Wykorzystanie elastycznego systemu zadań składającego się z Epiców, Historyjek (Story) oraz Podzadań (Subtasks).
- Wdrożenie automatyzacji w celu przyspieszenia powtarzalnych procesów, takich jak automatyczne zamykanie nadrzędnych zadań po ukończeniu przypisanych im podzadań.
- Mierzenie efektywności zespołu (Capacity/Velocity) za pomocą punktów Story Points.
- Usprawnienie komunikacji poprzez przeprowadzanie regularnych spotkań typu Daily Stand-up z wykorzystaniem tablicy oraz osi czasu w Jira.

2.2 Stos technologiczny

Zasadniczym narzędziem do realizacji procesu zarządzania projektem i dokumentowania przebiegu prac był system **Jira Software** w wersji cloud. Wykorzystano też funkcje raportowania natywnie wbudowane w narzędzie.

2.3 Oczekiwane rezultaty projektu

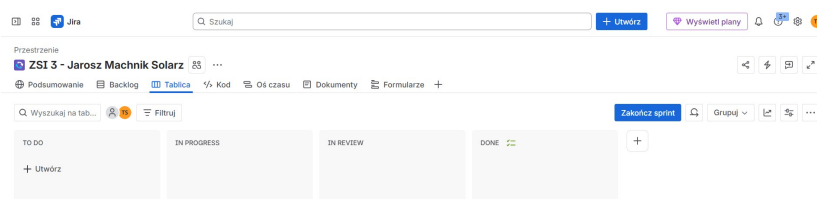
Oczekiwanym efektem końcowym jest zorganizowany, kompletny projekt w narzędziu Jira odzwierciedlający cykl życia oprogramowania poprzez trzy iteracje (Sprint 0, Sprint 1, Sprint 2). Rezultaty obejmują w szczególności zaplanowane wydania (Releases), poprawnie działające automatyzacje procesów oraz realne, adekwatne dane na wykresach raportowych (Burnup, Sprint Burndown, Velocity chart), potwierdzające sprawną realizację założonego harmonogramu.

3 Realizacja projektu

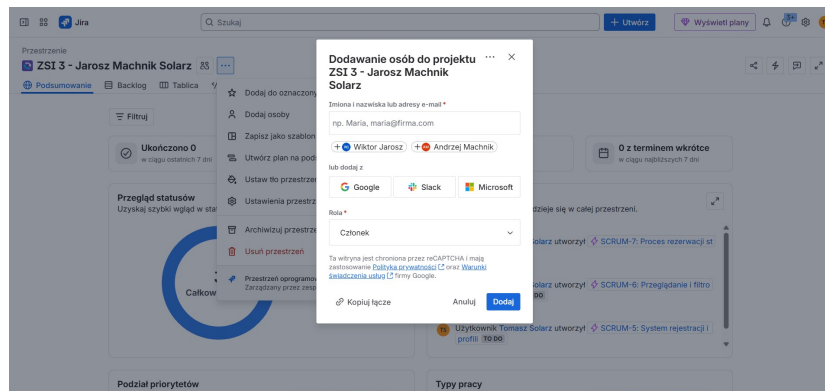
W ramach realizacji projektu pomyślnie przeszliśmy przez pełny cykl konfiguracji i prowadzenia projektu w systemie Jira. Poniżej przedstawiono poszczególne kroki i etapy wraz z ilustrującymi je zrzutami ekranu z wykonanej symulacji.

3.1 Konfiguracja początkowa i zespół

Na samym początku projektu skonfigurowano pustą tablicę Scrumową z podziałem na kolumny ułatwiające orientację w procesie: *To Do*, *In Progress*, *In Review* oraz *Done* (Rysunek 1). Następnie wysłano zaproszenia do pozostałych członków zespołu, tak by wszyscy mogli dołączyć do jednej, spójnej przestrzeni roboczej projektu (Rysunek 2).



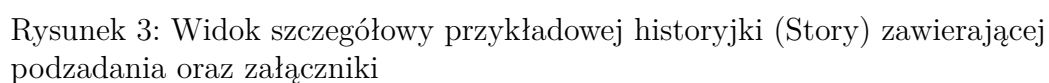
Rysunek 1: Skonfigurowana pusta tablica projektowa z podziałem na kolumny

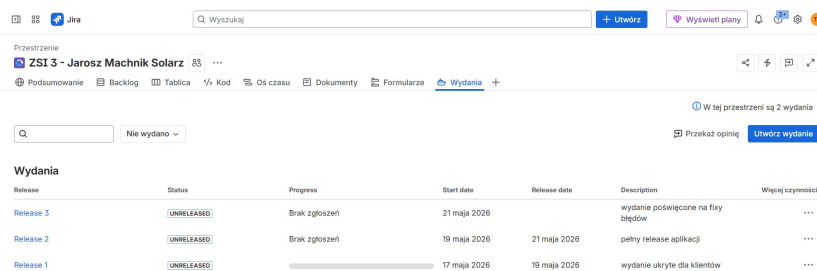


Rysunek 2: Wysyłanie zaproszeń umożliwiających dołączenie do projektu reszcie zespołu

3.2 Struktura zadań i planowanie wydań

Każda historyjka (Story) definiowana w projekcie posiadała szczegółowy opis, stosowne załączniki (na przykład makiety widoków ekranu), podzadania rozpisane na różne etapy realizacji oraz jasno zdefiniowany czas startu i termin wykonania (Rysunek 3). Zaplanowano również kilka odrębnych wydań (Releases), dla których ściśle określono daty, szerszy opis oraz zakres historyjek i zadań wchodzących w skład poszczególnej wersji (Rysunek 4).

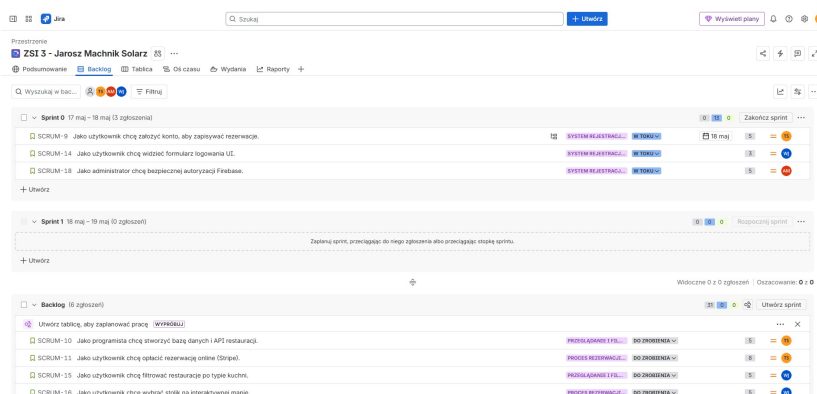




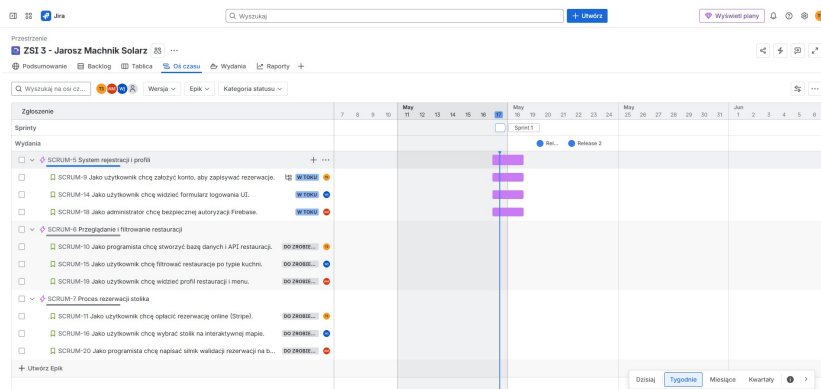
Rysunek 4: Zakładka wydań z rozplanowaniem planowanych wersji projektu

3.3 Backlog i planowanie Sprintów

Naszą pracę w znacznej mierze opieraliśmy o Backlog produktu. Zawarte w nim zadania były estymowane i wyceniane za pomocą miary Story Points (SP), a po wspólnej ocenie przydzielane do poszczególnych sprintów. Na Rysunku 5 widoczny jest przykładowy widok zadań, które zespół wziął do sprintu o łącznej zsumowanej wycenie na 13 SP. Z kolei zakładka z osią czasu (Timeline) pozwalała nam na intuicyjną wizualizację, które zadania ostatecznie znalazły się w danym sprincie wraz z oznaczeniem ich bieżącego statusu (Rysunek 6).



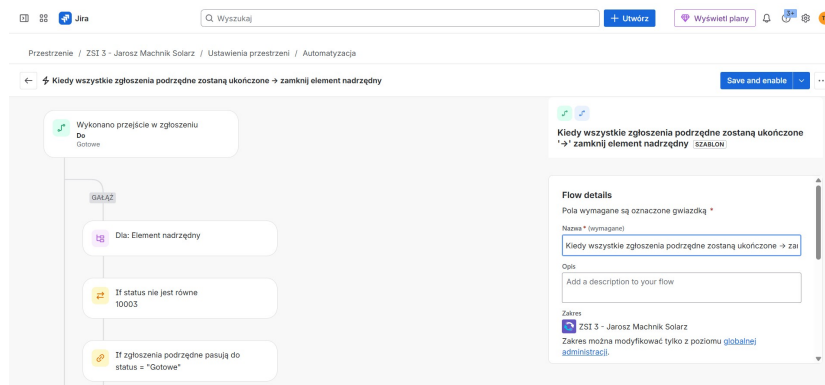
Rysunek 5: Widok Backlogu i zadań zaplanowanych do aktualnego sprintu na sumę 13 SP



Rysunek 6: Zakładka Timeline z widokiem zadań i historyjek przyjętych do bieżącego sprintu

3.4 Automatyzacje

Aby zminimalizować czynności manualne i usprawnić pracę zespołu, wprowadziliśmy systemową automatyzację, która samoczynnie zamyka danego Epica tuż po ukończeniu wszystkich przynależących podzadań. Zbudowano regułę w oparciu o silnik Automation for Jira: wyzwalaczem (Trigger) była zmiana statusu podzadania, a jako warunek (Condition) ustawiono weryfikację, czy wszystkie pozostałe podzadania powiązane z nadrzędnym Epicem znajdują się już w statusie *Done*. Akcją (Action) była natychmiastowa tranzycja zadania nadrzędnego. Na Rysunku 7 widnieje podgląd interfejsu konfiguracyjnego podczas tworzenia tej reguły. Poprawność jej działania potwierdza widoczny wpis w logu audytu (Rysunek 8), na którym zarejestrowano automatyczne przeniesienie historyjki do statusu *Gotowe*. Ponadto, system generował powiadomienia e-mail potwierdzające zamknięcie historyjki dla członków zespołu (Rysunek 9).



Rysunek 7: Proces tworzenia reguły automatyzacji domykającej zrealizowane Epici

Data i godzina	Identyfikator zdarzenia audytu	Flow	Status	Łączny czas
17.05.2026, 16:27:30	a58ac4df-6469-47d8-9e49-70913c843e45	Subtaski gotowe -> Zamknij Story (z opóźnieniem)	Done	2.76 s

✓ Wykonano przejście w zgłoszeniu 17.05.2026, 16:27:30 >

✓ Warunek pól zgłoszenia 17.05.2026, 16:27:30 >

✓ GAAZ >

✓ Branch flow / related work items 17.05.2026, 16:27:30 >

✓ SCRLM-9: Jako użytkownik chcę założyć konto, aby zapisywać recenzje. [Done]

✓ Pobierz ponownie dane zgłoszenia 17.05.2026, 16:27:31 >

Ponownie pobranie danych zgłoszenia z serwera powiodło się.

✓ SCRLM-9: Jako użytkownik chcę założyć konto, aby zapisywać recenzje. [Done]

✓ SCRLM-12: Skonfigurowanie formularza rejestracji użytkownika. [Done]

✓ Warunek powiązanych zgłoszeń 17.05.2026, 16:27:31 >

Następujące zgłoszenia spełniły warunki:

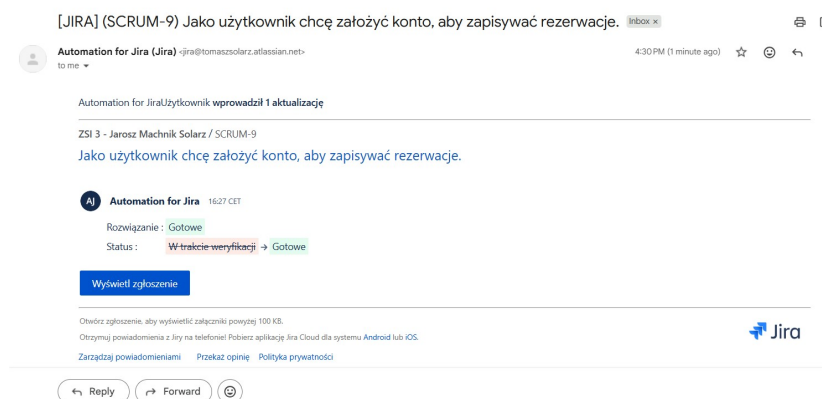
SCRLM-9

✓ Wykonaj przejście zgłoszenia 17.05.2026, 16:27:31 >

Zmieniono status zgłoszeń na:

Done

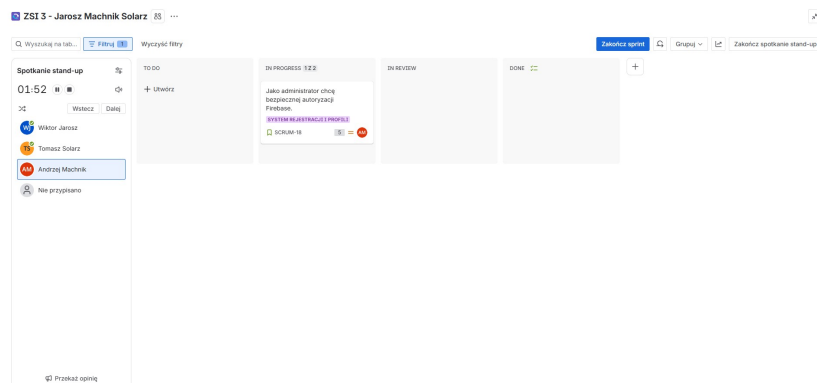
Rysunek 8: Log audytu weryfikujący poprawność działania skonfigurowanej automatyzacji



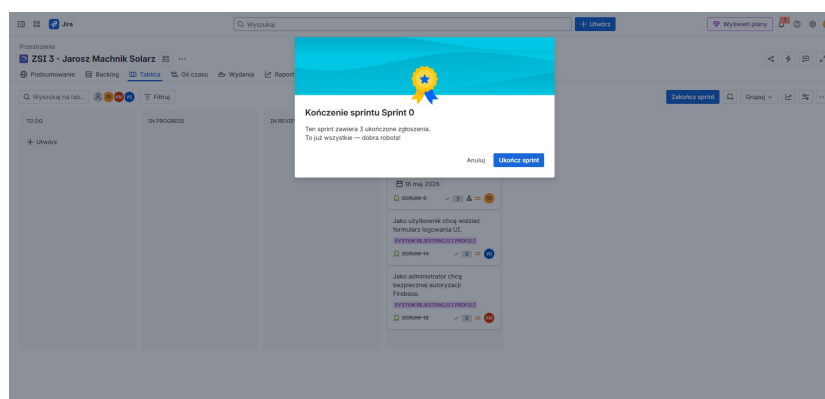
Rysunek 9: Powiadomienie mailowe wygenerowane z systemu Jira w wyniku działania automatyzacji

3.5 Przebieg Sprintów i spotkania codzienne (Stand-up)

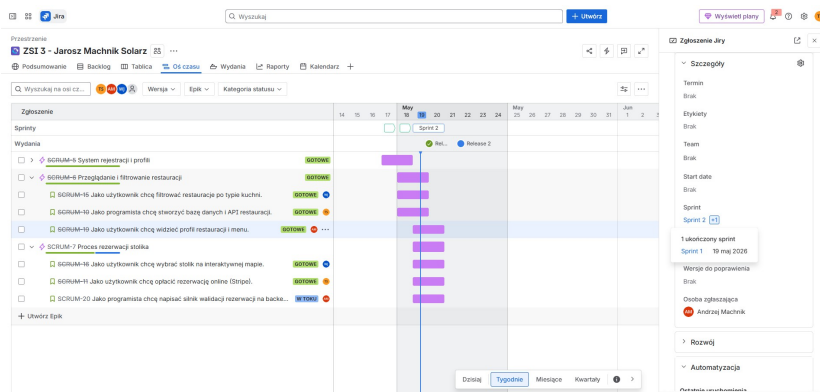
W trakcie pracy nad sprintami regularnie odbywały się zsynchronizowane spotkania Daily Stand-up. Widok z przykładowego spotkania, na którym raportowano przebieg zadań poszczególnych deweloperów (na Rysunku 10 widać perspektywę zadań Andrzeja, przy czym Wiktor i Tomasz zdążyli już zabrać głos), prezentował aktualny status i pozwalał koordynować dalsze prace. Z kolei Rysunek 11 przedstawia statystyki zaraz po całkowitym ukończeniu pierwszego sprintu weryfikacyjnego (tzw. Sprint 0). W toku projektu zdarzały się naturalnie sytuacje, w których nie wszystkie zadania udawało się dostarczyć na czas jak zadanie SCRUM-19 zaplanowane na Sprint 1 zostało przeniesione do Sprintu 2 z powodu opóźnień (Rysunek 12).



Rysunek 10: Podgląd tablicy filtrowanej pod kątem wybranego dewelopera podczas spotkania Stand-up



Rysunek 11: Podsumowanie i ukończenie wstępnego Sprintu 0



Rysunek 12: Oś czasu ukazująca zadanie nieukończone w terminie i przeniesione do kolejnego sprintu

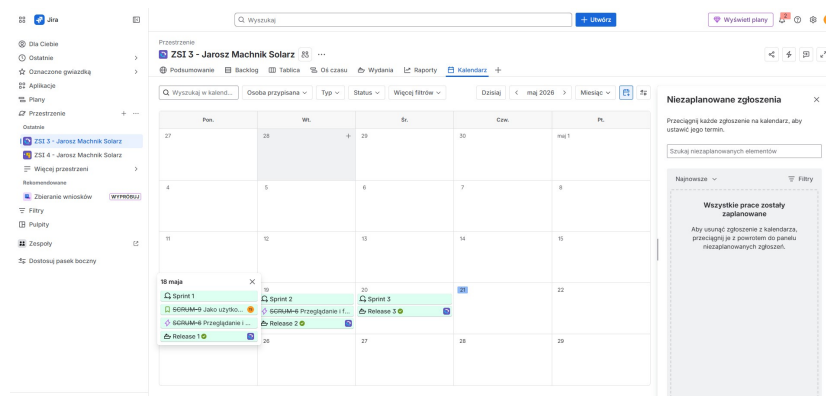
3.6 Raportowanie i analiza efektywności

Jira Software zapewniła rozbudowany wachlarz narzędzi do raportowania, co pozwalało realnie ocenić tempo pracy zespołu. Zakładka Kalendarza (Rysunek 13) dostarczała syntetycznego widoku na harmonogram odbytych sprintów, wydanych releasów oraz wskazywała konkretne dni ukończenia historyjek.

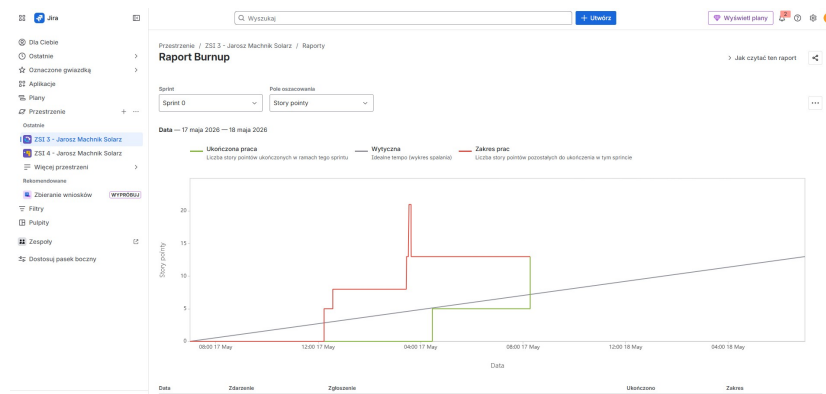
Wykres typu Burnup dla Sprintu 0 (Rysunek 14) rzetelnie obrazuje zmiany w zakresie prac na przestrzeni czasu sprintu. Pionowy, czerwony skok linii na tym wykresie reprezentuje sytuację, kiedy niedoświadczony deweloper omyłkowo dobrał do sprintu więcej zadań niż wynosiło tzw. capacity zespołu (13 SP), co następnie wymagało ręcznej korekty i interwencji ze strony Scrum Mastera. Z kolei wykres spalania sprintu (Burndown) pochodzący ze Sprintu 2 (Rysunek 15) ukazuje optymalne tempo wykonywania zadań w danym okresie czasu, na którym widoczne jest systematyczne ubywanie prac.

Ostateczną weryfikacją skuteczności planowania zespołu był ogólny Raport o szybkości pracy (Velocity chart). Zobrazował on jasny stosunek wartości Story Points początkowo zadeklarowanych (szary słupek) do tych ostatecznie dostarczonych i uznanych za gotowe (zielony słupek). Na Rysunku 16 wyraźnie widać trudności estymacyjne w Sprincie 1, w którym zespołowi nie udało się dostarczyć wszystkiego, do czego się zobowiązał. Wykres ukazuje, że w Sprincie 0 pomyślnie dostarczono zadeklarowane 13 SP. Z kolei w Sprincie 1 nastąpił spadek efektywności wynikający ze zbyt optymistycznego zaplanowania prac. Wykres w naturalny sposób wyliczał uśrednione tempo dostarczania SP na przestrzeni odbytych iteracji. Całościowy, szeroki podgląd wszystkich realizowanych Epiców, wydań i odbytych prac na prze-

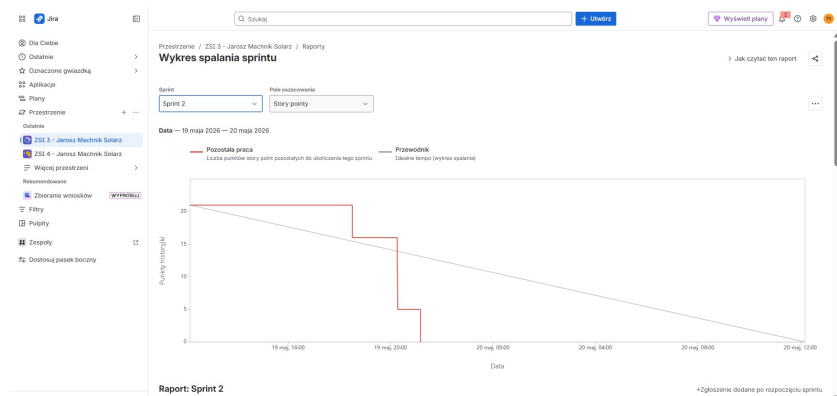
strzeni trzech sprintów ukazuje z perspektywy czasu główny widok Timeline (Rysunek 17).



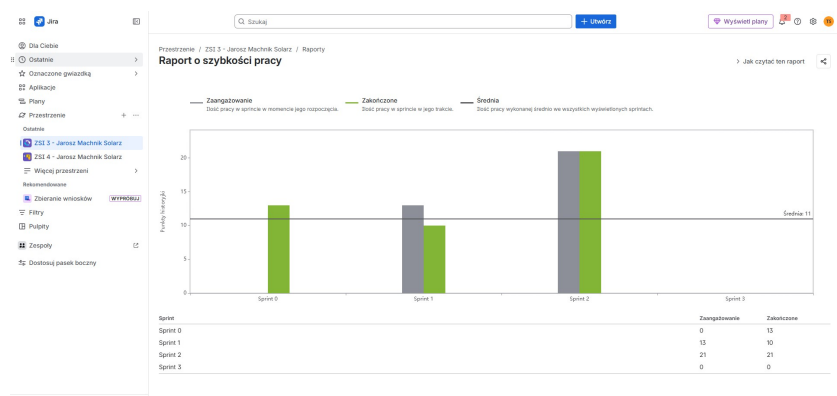
Rysunek 13: Widok zakładki kalendarza prezentujący ukończone epiki, historię oraz poszczególne sprinty



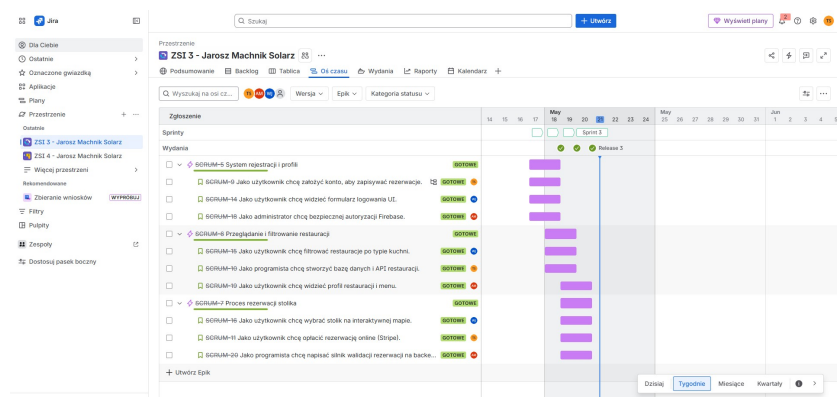
Rysunek 14: Raport Burnup ze Sprintu 0 ukazujący zakres prac i moment omyłkowego dołożenia zadań



Rysunek 15: Raport z wykresem spalania sprintu (Burndown chart) w trakcie Sprintu 2



Rysunek 16: Raport szybkości pracy zespołu (Velocity chart) z wyliczoną średnią



Rysunek 17: Oś czasu (Timeline) podsumowująca całościową pracę w trzech sprintach nad trzema wydaniem

4 Wnioski

- **Spostrzeżenia:** Aplikacja Jira to potężne narzędzie, które rewelacyjnie ułatwia śledzenie postępów prac oraz transparentną organizację. Zastosowanie w pełni wirtualnej tablicy redukuje koszty komunikacji, ale z drugiej strony wymaga dyscypliny i zrozumienia procesów przez cały zespół. Znaczące okazały się tu wnioski wyciągnięte chociażby po omyłkowym dodaniu zadań przez dewelopera w Sprincie 0, czy też przeszacowaniu możliwości zespołu w Sprincie 1.
- **Osiągnięcia:** Zespół z powodzeniem i zgodnie z planem wdrożył i przeprowadził symulowany cykl 3 iteracji (sprintów). Osiągnięto wymierną przewidywalność pracy, stabilizując Velocity zespołu po problemach początkowych (np. pomyślnie dostarczenie 13 SP w Sprincie 0 oraz korekty planistyczne w kolejnych sprintach). Sukcesem było skonfigurowanie i wykorzystanie automatyzacji pozwalającej oszczędzać czas na powtarzalnych zadaniach weryfikacji. Oprócz tego na bieżąco analizowano postępy prac przy użyciu zaawansowanych raportów (Velocity, Sprint Burndown, Burnup), co na każdym kroku podnosiło świadomość członków zespołu w kwestii capacity.
- **Potencjał rozwoju:** Istnieje wiele dróg na ulepszenie stosowanych procesów. W kolejnych krokach rozbudowy struktury zarządzania warto byłoby zintegrować narzędzie Jira z używanym systemem kontroli wersji (takim jak GitHub lub GitLab). Istniałaby wtedy możliwość powiązania kodu źródłowego bezpośrednio z ticketami w backlogu. Skrypty automatyzacji można by także rozszerzyć o automatyczne asygnowanie konkretnych recenzentów po przeniesieniu zadania do kolumny *In Review*.

5 Bibliografia

1. *Dokumentacja użytkownika i baza wiedzy Atlassian Jira* (<https://confluence.atlassian.com/jira>)
2. *The Scrum Guide* (<https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf>), Schwaber K., Sutherland J.